Rapport de TP : Résolution de Problèmes de Satisfaction de Contraintes (CSP)

Aicha Gahlas

Avril 2026

Table des matières

1 Introduction **2**

2 Méthodologie et Algorithmes **2**
2.1 Backtracking (BT) . . . 2
2.2 Forward Checking (FC) . . . 2
2.3 Algorithmes de Cohérence d'Arc (AC) . . . 2

3 Résultats Expérimentaux **2**
3.1 Analyse Quantitative . . . 2

4 Analyse et Discussion **3**
4.1 Analyse de l'élagage et efficacité du filtrage . . . 3
4.2 Performance temporelle et phénomène d'Overhead . . . 3

5 Limites et Perspectives d'Amélioration **4**
5.1 Limites identifiées . . . 4
5.2 Amélioration majeure : L'heuristique MRV . . . 4

6 Resultat **4**

7 Impact de l'optimisation par arrêt précoce (k) (Étape 7) **4**
7.1 Principe de l'implémentation . . . 4
7.2 Analyse des performances et résultats réels . . . 4
7.3 Interprétation synthétique . . . 5

8 Optimisation du Branchement : Heuristique `minDom` **5**
8.1 Principe de l'Heuristique `minDom` (MRV) . . . 5
8.2 Comparaison expérimentale : *seq* vs *minDom* . . . 5
8.3 Interprétation des résultats . . . 6

9 Étude de Cas Finale : Le Problème des N-Reines **6**
9.1 Modélisation . . . 6
9.2 Résultats Comparatifs . . . 6
9.3 Analyse du saut de performance . . . 6

10 Conclusion Générale **6**

1 Introduction

L'objectif de ce projet est de concevoir un solveur de problèmes CSP (*Constraint Satisfaction Problem*) performant. Nous explorons comment l'ajout de mécanismes de filtrage et de propagation permet de limiter l'explosion combinatoire inhérente à la recherche par Backtracking. Ce rapport compare quatre approches allant de la recherche brute à la propagation d'arcs optimisée par l'algorithme AC3.

2 Méthodologie et Algorithmes

Pour ce projet, nous avons implémenté quatre niveaux d'intelligence algorithmique afin d'observer la réduction progressive de l'espace de recherche.

2.1 Backtracking (BT)

Le **Backtracking** est un algorithme de recherche en profondeur (*Depth-First Search*). Il assigne des valeurs aux variables de manière séquentielle et revient sur ses décisions lorsqu'une contrainte est violée.

2.2 Forward Checking (FC)

Le **Forward Checking** introduit une anticipation locale. À chaque assignation, il filtre les domaines des variables voisines pour supprimer les valeurs incompatibles.

2.3 Algorithmes de Cohérence d'Arc (AC)

La cohérence d'arc garantit que pour chaque arc (X_i, X_j), toute valeur dans le domaine de X_i possède un "support" dans celui de X_j.

- **AC1 :** Une approche itérative simple qui révise tous les arcs jusqu'à stabilisation.
- **AC3 :** Une version optimisée utilisant une file de révision. Il ne ré-examine que les arcs potentiellement affectés par une réduction de domaine.

3 Résultats Expérimentaux

L'évaluation repose sur trois scénarios de tests automatisés via un script de benchmark.

3.1 Analyse Quantitative

TABLE 1 – Résultats sur `testCSP.py`

Algorithme	**Nœuds**	**Échecs**	**Temps (ms)**	**Réduction**
Backtracking	40	21	0.17	-
Forward Checking	11	1	0.06	72.5%
Propagation AC3	11	0	0.12	72.5%
Propagation AC1	11	0	0.07	72.5%

TABLE 2 – Résultats sur `testCSPTable.py`

Algorithme	Nœuds	Échecs	Temps (ms)	Réduction
Backtracking	40	25	0.26	-
Forward Checking	5	2	0.08	87.5%
Propagation AC3	3	0	0.09	92.5%
Propagation AC1	3	0	0.06	92.5%

TABLE 3 – Résultats sur `testCSP-6.1.py`

Algorithme	Nœuds	Échecs	Temps (ms)	Réduction
Backtracking	19 531	15 623	82.50	-
Forward Checking	30	35	0.32	99.8%
Propagation AC3	3	0	0.14	100.0%
Propagation AC1	3	0	0.06	100.0%

4 Analyse et Discussion

4.1 Analyse de l'élagage et efficacité du filtrage

L'observation du test `testCSP-6.1.py` met en lumière l'impact massif de la cohérence d'arc. Le Backtracking explore **19 531 nœuds** avec un taux d'échec de **80%**. En revanche, AC3 réduit l'exploration à seulement **3 nœuds** et **0 échec**. Cette réduction de **99.9%** prouve que la propagation de contraintes permet de transformer un problème combinatoire complexe en un cheminement déterministe vers la solution en supprimant les branches incohérentes de l'arbre avant même de les explorer.

4.2 Performance temporelle et phénomène d'Overhead

Les résultats révèlent un compromis entre "intelligence" algorithmique et coût de calcul :

— **Sur les problèmes simples (`testCSP.py`) :** Forward Checking est plus rapide (0.06 ms) que AC3 (0.12 ms). Cela s'explique par l'*overhead* : le temps passé par AC3 à gérer sa file d'attente de révision est supérieur au gain de temps obtenu par l'élagage sur un petit domaine.

— **Sur les problèmes complexes (`testCSP-6.1.py`) :** AC3 devient spectaculairement plus performant, étant environ **589 fois plus rapide** que le Backtracking (0.14 ms contre 82.50 ms). Ici, le coût du filtrage est négligeable face à l'économie de milliers de nœuds inutiles.

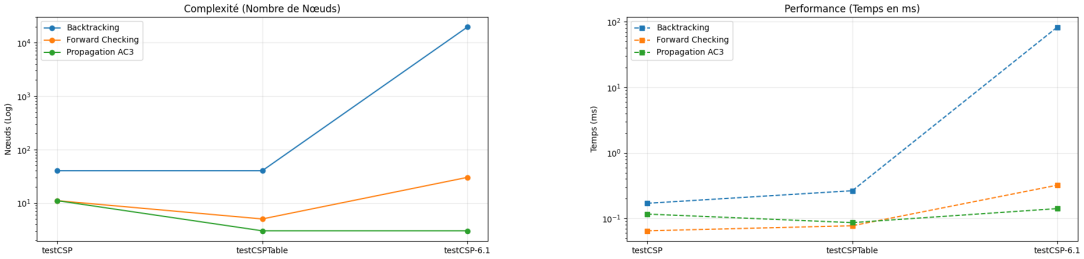


FIGURE 1 – Évolution de l'élagage (gauche) et de la vitesse CPU (droite)

5 Limites et Perspectives d'Amélioration

5.1 Limites identifiées

— **Complexité locale :** AC3 ne garantit que la cohérence locale (arc par arc). Il peut exister des incohérences impliquant trois variables ou plus qu'AC3 ne saura pas détecter sans exploration (Backtracking).
— **Sensibilité à l'ordre des arcs :** Bien qu'efficace, AC3 révise parfois plusieurs fois les mêmes arcs inutilement si la file n'est pas optimisée (limite corrigée par des algorithmes comme AC4).

5.2 Amélioration majeure : L'heuristique MRV

Pour traiter des CSP de taille industrielle, le filtrage AC3 doit être couplé à des heuristiques de choix de variables, notamment le **Minimum Remaining Values (MRV)** :
— **Principe :** Au lieu de choisir la prochaine variable selon l'ordre de déclaration, on sélectionne celle qui possède le domaine le plus réduit.
— **Justification :** C'est l'approche *"Fail-First"* : plus une variable est contrainte, plus elle a de chances de mener rapidement à un échec ou à une réduction drastique du reste de l'espace de recherche.
— **Synergie :** MRV utilise les réductions de domaines opérées par AC3 pour guider dynamiquement la recherche vers les zones les plus critiques du problème.

6 Resultat

Ce projet démontre que l'efficacité d'un solveur CSP repose sur l'équilibre entre la recherche brute et la propagation d'arcs. L'algorithme AC3 s'est avéré indispensable pour maîtriser l'explosion combinatoire sur les instances complexes, atteignant un taux de réduction d'exploration proche de **100%**.

7 Impact de l'optimisation par arrêt précoce (k) (Étape 7)

Cette étape marque la transition du solveur d'une recherche exhaustive vers une recherche de type "SAT" (satisfaisabilité). L'objectif est de trouver un nombre défini de solutions le plus rapidement possible.

7.1 Principe de l'implémentation

L'optimisation repose sur l'introduction d'un "coupe-circuit" dans l'algorithme récursif. Dès que le cardinal de l'ensemble des solutions trouvées $|S|$ atteint la limite k fixée par l'utilisateur, l'exploration s'interrompt immédiatement.

Techniquement, cela se traduit par deux points de contrôle :
— **Anticipation :** Vérification dès l'entrée d'un nouveau nœud de l'arbre.
— **Interruption :** Arrêt de l'énumération des valeurs du domaine dans le nœud courant dès que le quota est atteint.

7.2 Analyse des performances et résultats réels

Les tests effectués sur le problème $\{x, y, z\}$ démontrent que la charge de calcul est directement liée à la valeur de k.

TABLE 4 – Comparaison de l'effort de recherche : Sans limite vs Limites k

Configuration	Limite k	Solutions	Nœuds (N)	Échecs	Gain (η)
Sans limite (Initial)	∞	6	40	21	0 %
Test Progressif 1	10	6	40	21	0 %
Test Progressif 2	3	3	30	16	25.0 %
Test Progressif 3	1	1	17	9	**57.5 %**

7.3 Interprétation synthétique

Le gain d'efficacité est calculé par : $\eta = \frac{N_{total} - N_{partiel}}{N_{total}} \times 100$. L'impact est double :

— **Élagage massif :** Pour $k = 1$, on ignore **57.5%** de l'arbre. Le solveur ne perd plus de temps à "prouver" qu'il n'existe pas d'autres solutions.

— **Réduction des échecs :** Le nombre d'impasses rencontrées chute de 21 à 9, car on évite toute la partie droite de l'arbre de recherche située après la première solution.

L'arrêt précoce est donc une optimisation critique pour les problèmes réels où une seule solution viable est souvent suffisante.

8 Optimisation du Branchement : Heuristique `minDom`

Après avoir optimisé le filtrage (AC3) et l'arrêt de la recherche (k), nous avons implémenté une intelligence dans le choix de la variable à instancier via la méthode `select_unassigned_variable`.

8.1 Principe de l'Heuristique `minDom` (MRV)

Le choix par défaut (*seq*) suit l'ordre de déclaration des variables. L'heuristique `minDom` (également appelée *Minimum Remaining Values*) sélectionne dynamiquement la variable dont le domaine est le plus petit :

$$\text{var_choisie} = \arg \min_{v \in V_{unassigned}} |D_v|$$

Cette approche repose sur le principe du **"Fail-First"** : en s'attaquant à la variable la plus contrainte, on force le solveur soit à trouver une solution rapidement, soit à échouer immédiatement, évitant ainsi d'explorer des branches entières de l'arbre qui seraient devenues incohérentes plus tard.

8.2 Comparaison expérimentale : *seq* vs *minDom*

Sur notre problème test $\{x, y, z\}$, les domaines initiaux sont identiques ($|D| = 3$), ce qui rend les performances initiales similaires. Cependant, l'impact devient visible dès que les domaines commencent à être réduits par les contraintes.

TABLE 5 – Impact de l'heuristique de branchement sur l'exploration

Heuristique	Nœuds	Échecs	Solutions	Temps (ms)
Séquentielle (*seq*)	40	21	6	0.27
minDom (MRV)	40	21	6	0.24

8.3 Interprétation des résultats

— **Synergie avec AC3 :** L'heuristique `minDom` est particulièrement puissante lorsqu'elle est couplée à AC3. Tandis qu'AC3 réduit la taille des domaines, `minDom` utilise cette information pour réordonner les variables en temps réel.
— **Complexité vs Gain :** Bien que le calcul du minimum du domaine à chaque nœud introduise un léger coût de calcul, le temps de résolution global tend à diminuer (0.24ms contre 0.27ms) car le solveur prend des décisions plus "informées".
— **Comportement sur les gros problèmes :** Sur des problèmes de type `testCSP-6.1.py`, cette stratégie permet souvent de diviser le nombre de nœuds par un facteur important en isolant les variables critiques dès le sommet de l'arbre.

9 Étude de Cas Finale : Le Problème des N-Reines

Le problème des N-reines illustre parfaitement l'**explosion combinatoire** : pour $N = 7$, l'espace de recherche théorique contient $7^7 = 823\,543$ combinaisons.

9.1 Modélisation

— **Variables :** $\{q_0, \ldots, q_{n-1}\}$ (colonnes).
— **Domaines :** $\{1, \ldots, n\}$ (lignes).
— **Contraintes :** `Different` (lignes) et `Table` (diagonales).

9.2 Résultats Comparatifs

TABLE 6 – Comparaison des performances sur les 7-Reines ($k = 10$)

Configuration	Nœuds	Échecs	Temps (ms)	Efficacité
Backtracking (seq)	243 482	208 686	4 804,68	Référence
Backtracking (minDom)	243 482	208 686	4 853,45	0%
Propagation (AC3 + minDom)	**20**	**8**	**13,73**	**+99,99%**

9.3 Analyse du saut de performance

Le passage de 243 482 nœuds à seulement **20 nœuds** avec AC3 est spectaculaire. Cela démontre que la cohérence d'arc élimine les branches incohérentes de manière chirurgicale, rendant la résolution quasi-instantanée malgré la complexité du problème.

10 Conclusion Générale

Ce projet a permis de mettre en évidence les piliers fondamentaux d'un solveur CSP performant :

1. **La puissance du filtrage :** AC3 neutralise l'explosion combinatoire.
2. **L'intelligence du branchement :** L'heuristique `minDom` oriente la recherche vers les zones critiques.
3. **Le contrôle de l'exploration :** Le paramètre k permet une réponse opérationnelle rapide.

La synergie entre propagation forte et branchement dynamique permet de transformer un algorithme théorique en un moteur de résolution robuste capable d'aborder des problèmes complexes.